

EduTrust-Africa: A Decentralized Solution for Cross-Border Academic Credential Verification

Tamunotonye Briggs

GitHub Link - <https://github.com/TammyBriggs/EduTrust-Africa/tree/main>

Demo Link -  EduTrust-Africa Final Summative Demo

Problem Identification and Alignment with Mission

The spread and increase of fraudulent academic credentials and the fragmented nature of university databases present a severe crisis of trust within the African education sector. Currently, verifying a degree across borders, such as an employer in Nigeria attempting to authenticate a transcript from a university in Rwanda, relies on slow, manual, and centralized institutional mediation. This creates bottlenecks for graduates and exposes employers to sophisticated forgery.

My personal mission as a software engineer is to bridge gaps in systems, access, and trust by building practical, secure, and human-centered technology that creates meaningful impact within the African continent majorly, and the world as a whole. EduTrust-Africa aligns perfectly with this mission. By decentralizing the verification process, it eliminates the "trust gap" between borders, providing a secure, tamper-proof, and accessible infrastructure for academic mobility.

Solution Design

EduTrust-Africa is a decentralized Web3 application (dApp) utilizing Ethereum smart contracts to provide an immutable, cross-border credential registry.

- **Key Features:**
 - **Browser-Side Cryptography:** Student certificates (PDFs) are hashed locally via SHA-256 in the browser, ensuring data privacy. Only the hash is anchored to the blockchain.
 - **Accredited Issuer Registry:** A strict access-control system ensuring only verified institutions can mint or revoke credentials.
 - **Instant Public Verification:** Employers can verify a document's authenticity and view its Sepolia Etherscan transaction receipt without needing a crypto wallet or institutional login.
- **Role of the Smart Contract:** The Solidity contract acts as the ultimate enforcer of truth. It maps cryptographic document hashes to authorized institutional wallet addresses, rendering forgery mathematically impossible and eliminating reliance on centralized university servers.

Smart Contract Development

The core logic was developed in Solidity (pragma solidity 0.8.28;).

- *EduTrust.sol code snippet.*

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

/**
 * @title EduTrust Africa Credential Verifier
 * @dev A smart contract for issuing, verifying, and revoking
cross-border academic credentials.
 */
contract EduTrust {
    address public admin;

    struct Issuer {
        string institutionName;
        string country; // e.g., "Nigeria", "Rwanda"
        bool isAuthorized;
    }

    struct Credential {
        bytes32 credentialHash; // The off-chain document hash
        address issuerAddress;
        string studentIdHash; // Privacy-preserving identifier
        string programName;
        uint256 issueDate;
        bool isValid;
        bool exists;
    }

    // Mappings
    mapping(address => Issuer) public accreditedIssuers;
    mapping(bytes32 => Credential) public credentials;

    // Events
    event IssuerAuthorized(address indexed issuerAddress, string
institutionName, string country);
    event IssuerRevoked(address indexed issuerAddress);
    event CredentialIssued(bytes32 indexed credentialHash, address
indexed issuerAddress);
    event CredentialRevoked(bytes32 indexed credentialHash, address
```

```

indexed issuerAddress);
    event IssuerAccessRevoked(address indexed issuerAddress);

    // Modifiers
    modifier onlyAdmin() {
        require(msg.sender == admin, "EduTrust: Only admin can perform
this action");
        _;
    }

    modifier onlyAuthorizedIssuer() {
        require(accreditedIssuers[msg.sender].isAuthorized, "EduTrust:
Caller is not an authorized issuer");
        _;
    }

    constructor() {
        admin = msg.sender;
    }

```

- **Explanation:** The contract utilizes struct mappings (accreditedIssuers and credentials) to manage state. Modifiers (onlyAdmin, onlyAuthorizedIssuer) enforce strict governance. The logic handles credential issuance, status verification, and revocation to reflect real-world academic administration.

Deployment Guide

Prerequisites & Setup

1. Ensure Node.js and npm are installed.
2. Initialize a Hardhat project: `npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox dotenv`
3. Secure the environment: Create a .gitignore file to prevent the upload of sensitive configuration files. Create a .env file containing your SEPOLIA_RPC_URL, MetaMask PRIVATE_KEY, and ETHERSCAN_API_KEY.

Deploying to the Ethereum Sepolia Testnet

1. **Compile the Contract:** Run `npx hardhat compile` to ensure the Solidity code is error-free.
2. **Execute Deployment:** Run the deployment script using `npx hardhat run scripts/deploy.js --network sepolia`.
3. **Record Address:** Copy the deployed contract address output in the terminal.
4. **Verify on Etherscan (Optional but recommended):** Run `npx hardhat verify --network sepolia <DEPLOYED_ADDRESS>` to make the contract code publicly readable.

5. **Connect Frontend:** Paste the deployed address into the React frontend (App.jsx) `CONTRACT_ADDRESS` constant to bridge the UI with the blockchain.

Screenshots showing successful deployment and interaction with the smart contract

The smart contract was deployed successfully, and the contract address displayed

```
◇ injected env (3) from .env // tip: ◇ encrypted .env [www.dotenvx.com]
Compiled 1 Solidity file successfully (evm target: paris).
◇ injected env (0) from .env // tip: ⚙️ custom filepath { path: '/custom/path/.env' }
-----
Deploying EduTrust Africa Smart Contract...
-----
✅ EduTrust deployed successfully!
📌 Contract Address: 0x4C117612Fa38CD8E2B20072D7D233aB54e1f9868
-----
Next steps:
1. Verify contract on Etherscan:
   npx hardhat verify --network sepolia 0x4C117612Fa38CD8E2B20072D7D233aB54e1f9868
-----
```

Smart contract successfully verified on the block explorer

```
Successfully submitted source code for contract
contracts/EduTrust.sol:EduTrust at 0x4C117612Fa38CD8E2B20072D7D233aB54e1f9868
for verification on the block explorer. Waiting for verification result...

Successfully verified contract EduTrust on the block explorer.
https://sepolia.etherscan.io/address/0x4C117612Fa38CD8E2B20072D7D233aB54e1f9868#code
```

Validation and Testing

Testing was conducted using Hardhat, Chai, and Ethers.js, resulting in 17 passing integration tests simulating real-world cross-border verification scenarios.

```
◇ injected env (0) from .env // tip: %enable debugging { debug: true }

EduTrust Africa Smart Contract
  Deployment & Administration
    ✓ Should set the right admin
    ✓ Should allow the admin to authorize an issuer
    ✓ Should revert if a non-admin tries to authorize an issuer
  Credential Issuance & Verification
    ✓ Should allow an authorized issuer to mint a credential
    ✓ Should revert if an unauthorized wallet tries to issue a credential
    ✓ Should revert if trying to verify a non-existent credential
  Credential Revocation
    ✓ Should allow the original issuer to revoke a credential
    ✓ Should prevent a different issuer from revoking the credential

Lock
  Deployment
    ✓ Should set the right unlockTime
    ✓ Should set the right owner
    ✓ Should receive and store the funds to lock
    ✓ Should fail if the unlockTime is not in the future
  Withdrawals
    Validations
      ✓ Should revert with the right error if called too soon
      ✓ Should revert with the right error if called from another account
      ✓ Shouldn't fail if the unlockTime has arrived and the owner calls it
    Events
      ✓ Should emit an event on withdrawals
    Transfers
      ✓ Should transfer the funds to the owner

17 passing (801ms)
```

Test Case	Expected Outcome	Actual Result
Admin Setup	The deployer wallet is successfully set as the network Admin.	Passed (Admin assigned correctly)
Authorize Issuer	Admin can successfully add an institution to the accreditedIssuers mapping.	Passed (Issuer authorized)

Unauthorized Access	Non-admin wallets attempting to authorize institutions are reverted.	Passed (Reverted with "Only admin")
Mint Credential	An authorized issuer can successfully mint a new credential hash to the ledger.	Passed (Event CredentialIssued emitted)
Prevent Forgery	An unauthorized wallet attempting to mint a credential is blocked.	Passed (Reverted with "Not an authorized issuer")
Duplicate Minting	Issuing the exact same credential hash twice reverts the transaction.	Passed (Reverted with "Credential already exists")
Verify Valid Doc	Querying an existing hash returns isValid: true and the issuer's metadata.	Passed (Returns true + Institution details)
Verify Fake Doc	Querying a non-existent hash reverts the transaction.	Passed (Reverted with "Credential not found")
Revoke Credential	The original issuer can successfully revoke an active credential.	Passed (Status changes to isValid: false)
Cross-Revocation	Institution A cannot revoke a credential minted by Institution B.	Passed (Reverted with "Only original issuer can revoke")